



Práctica 4

Programación con MPI

Estructura general de programa para MPI:

```
fProcesoTipoA(){
// Función que implementa el comportamiento de los procesos de tipo A
}

fProcesoTipoB(){
// Función que implementa el comportamiento de los procesos de tipo B
}

fProcesoTipoC(){
// Función que implementa el comportamiento de los procesos de tipo C
}

int main(int argc, char* argv[]){
MPI_Init(&argc, &argv);
int id;
int cantidadDeProcesos;

    MPI_Comm_rank(MPI_COMM_WORLD,&id);
    MPI_Comm_size(MPI_COMM_WORLD,&cantidadDeProcesos);
...
    if(id == 0)
        fProcesoTipoA();
    else if (id >= 1 && id <= 3)
        fProcesoTipoB();
    else
        fProcesoTipoC();

    MPI_Finalize();
    return(0);
}
```

NOTA: El ejemplo de programa anterior supone que habrá tres tipos de procesos con tres comportamientos diferentes. El comportamiento del proceso con $id=0$ será implementado en la función `fProcesoTipoA()`. El comportamiento de los procesos con $id=1, 2$ y 3 será implementado en la función `fProcesoTipoB()`. El comportamiento de los procesos restantes con $id=4$ a `cantidadDeProcesos` será implementado en la función `fProcesoTipoC()`.

Pautas:

Compilar en Linux OpenMPI:

`mpicc -o salidaEjecutable archivoFuente`

Ejecutar en OpenMPI:

En una sola máquina:

mpirun -np cantidadDeProcesos ejecutable

En un cluster de máquinas:

mpirun -np cantidadDeProcesos -machinefile archivoMaquinas ejecutable

El formato de archivo de máquinas es:

maquina1 slots=cantidad de procesadores de la maquina1

maquina2 slots=cantidad de procesadores de la maquina2

...

maquinaN slots=cantidad de procesadores de la maquinaN

En todos los ejercicios implementar el algoritmo secuencial y el algoritmo paralelo.

Probar el algoritmo paralelo sobre:

- 1) *Una máquina con 4 procesos*
- 2) *Dos máquinas con 2 procesos por máquina (4 procesos)*
- 3) *Dos máquinas con 8 procesos.*

Calcular el Speedup y la eficiencia. Realizar el análisis de escalabilidad y analizar el overhead introducido por comunicación.

1. Resolver una multiplicación de matrices de $N \times N$ y analizar los tiempos de comunicación utilizando:
 - a) Operaciones punto a punto.
 - b) Operaciones colectivas
 Utilizar tamaños de matrices de 512, 1024 y 2048.
2. Realizar un algoritmo paralelo que dada una matriz A de $N \times N$ obtenga el valor máximo, el valor mínimo y valor promedio de A , luego debe armar una matriz B de la siguiente forma:
 - Si el elemento $a_{i,j} < \text{promedio}(A)$ entonces $b_{i,j} = \min(A)$.
 - Si el elemento $a_{i,j} > \text{promedio}(A)$ entonces $b_{i,j} = \max(A)$.
 - Si el elemento $a_{i,j} = \text{promedio}(A)$ entonces $b_{i,j} = \text{promedio}(A)$.
3. Dado un vector de N elementos enteros creciente, paralelizar un algoritmo que determine cada cuanto se duplica cada número de la serie. (-1 si no se duplica)
4. Realizar un algoritmo paralelo que ordene un vector de N elementos por mezcla.
5. Dado un texto representado por un vector T de tamaño N , se debe realizar un algoritmo paralelo que obtenga la lista de palabras de T , y luego determine la cantidad de veces que aparece cada palabra en el texto quedándose sólo con las cinco palabras más frecuentes.
6. Pensar un diseño y posible implementación al problema de las N Reinas sobre memoria distribuida utilizando MPI. ¿Cómo distribuir el trabajo cuando N es menor/mayor al número de procesos? Analizar distintos paradigmas.